

Explain Pair Difference Problem

Yes. Let's only understand what the question is asking, without jumping into the solution.

What are we given?

We have:

```
nums = [10, 1, 2, 7, 1, 3]
p = 2
```

We need to choose 2 pairs of indices.

For example:

```
(1, 4) → nums[1] = 1, nums[4] = 1
(2, 5) → nums[2] = 2, nums[5] = 3
```

So the differences are:

```
|1 - 1| = 0
|2 - 3| = 1
```

The maximum difference among these pairs is:

```
max(0, 1) = 1
```

But why are we doing this?

The question says:

Find p pairs such that the maximum difference among those pairs is as small as possible.

So we're allowed to choose any p pairs, but:

One index cannot be used more than once.

For example, this is invalid:

```
(1, 2)
(1, 4)
```

because index 1 is used twice.

Think of it as a game

Suppose:

```
nums = [10, 1, 2, 7, 1, 3]
p = 2
```

We need 2 pairs.

We want the pairs to have differences as small as possible.

For example:

```
Pair 1: (1,4)
values: 1,1
difference = 0
```

```
Pair 2: (2,5)
values: 2,3
difference = 1
```

Therefore:

```
maximum difference = 1
```

Could we make the maximum difference 0?

That would mean we need two pairs whose values are exactly equal.

We only have:

```
1 → two times
2 → one time
3 → one time
7 → one time
10 → one time
```

We can make one zero-difference pair:

```
(1,4) → 1,1
```

But we cannot make another zero-difference pair.

Therefore 0 is impossible.

So the minimum possible maximum difference is:

```
1
```

The key thing to understand

There are actually **two levels** of optimization:

Level 1: Choose p pairs.

Level 2: Among those pairs, look at the **largest difference**.

We want to minimize that largest difference.

Mathematically:

```
Choose p pairs
    ↓
calculate each pair's difference
    ↓
take the maximum difference
    ↓
make that maximum as small as possible
```

For example, if we choose pairs with differences:

```
[0, 1]
```

maximum = 1

If another selection gives:

```
[2, 3]
```

maximum = 3

We prefer the first one because:

```
1 < 3
```

So the answer is the **smallest possible value of that maximum**.

And importantly, **the pairs are based on indices**, but the cost/difference is based on their values.

The main approach here is **Sorting + Greedy + Binary Search on Answer**.

Let's understand *why* step by step.

1. First sort the array

For:

```
nums = [10, 1, 2, 7, 1, 3]
```

sort it:

```
[1, 1, 2, 3, 7, 10]
```

Why?

Because after sorting, the best pairs will always come from **nearby elements**.

For example:

```
1 1 2 3 7 10
↑ ↑
```

Difference = 0

and

```
2 3
↑ ↑
```

Difference = 1

These are naturally good candidates.

2. Binary search the answer

The question asks:

What is the minimum possible maximum difference?

That's a classic **binary search on answer** problem.

Suppose we ask:

"Can I create p pairs such that every pair has difference $\leq x$?"

For example, try:

```
x = 1
```

Meaning:

Can I make 2 pairs where every pair has difference at most 1?

For:

```
[1, 1, 2, 3, 7, 10]
```

we can greedily take:

```
(1,1) → difference 0
(2,3) → difference 1
```

Yes → we can make 2 pairs.

So answer could be 1 or smaller.

Try:

```
x = 0
```

Can we make 2 pairs where every difference ≤ 0 ?

Only:

```
(1,1)
```

So we can make only 1 pair.

Therefore 0 is impossible.

Hence answer = 1.

3. How do we check x ?

This is the **greedy part**.

After sorting:

```
[1, 1, 2, 3, 7, 10]
```

Start from left.

For every adjacent pair:

```
nums[i], nums[i+1]
```

If:

```
nums[i+1] - nums[i] <= x
```

then take them as a pair and move by 2 .

Otherwise, skip the current element and move by 1 .

For $x = 1$:

```
1 1 2 3 7 10
| |
pair → 1,1
```

then

```
2 3
| |
pair → 2,3
```

We got 2 pairs.

Overall flow

```
nums
↓
sort
↓
binary search minimum answer
↓
for each possible difference x
↓
greedily count how many pairs can be formed
↓
if pairs >= p
    x is possible
else
    x is impossible
```

The important function conceptually is:

```
canMakePairs(x)
```

It answers:

"Using maximum allowed difference x , can I create at least p non-overlapping pairs?"

If yes:

```
try smaller x
```

If no:

```
try larger x
```

So the final approach is:

Sorting → Binary Search on Answer → Greedy Pairing

If you want, next I can explain **exactly why the greedy adjacent-pair strategy is correct**, because that's the most important part to understand before writing the code.

Here is the clean **Sorting + Binary Search + Greedy** solution for LeetCode 2616:

```
class Solution {
public:
    bool canMake(vector<int>& nums, int p, int maxDiff) {
        int pairs = 0;

        for (int i = 0; i < nums.size() - 1; ) {
            if (nums[i + 1] - nums[i] <= maxDiff) {
                pairs++;
                i += 2;
            } else {
                i++;
            }
        }
    }
}
```

```

        if (pairs >= p)
            return true;
    }

    return false;
}

int minimizeMax(vector<int>& nums, int p) {
    sort(nums.begin(), nums.end());

    int low = 0;
    int high = nums.back() - nums.front();

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (canMake(nums, p, mid)) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }

    return low;
}
};

```

Flow on Example 1

```

nums = [10,1,2,7,1,3]
p = 2

```

After sorting:

```
[1,1,2,3,7,10]
```

Binary search asks:

```
Can I make 2 pairs with difference <= mid?
```

For mid = 1:

```

[1,1] → diff 0 → pair
[2,3] → diff 1 → pair

```

We got 2 pairs, so 1 is possible.

For mid = 0:

```

[1,1] → diff 0 → pair
[2,3] → diff 1 → cannot pair

```

Only 1 pair, so 0 is impossible.

Therefore:

```
answer = 1
```

Time complexity: $O(n \log n)$ for sorting + $O(n \log(\max(\text{nums}) - \min(\text{nums})))$ for binary search.